Tema 027 – APD1 + IS

File-Mitita Traian · Merge Sort paralel cu producător–consumator (BlockingCollection)

1. APD1 – Merge Sort cu BlockingCollection

Ideea. Vectorul se împarte în blocuri consecutive de **B = 512** elemente, care se sortează fiecare cu Merge Sort secvențial. Apoi blocurile se interclasează două câte două, pe niveluri (lățimea w = 512, 1024, 2048, …). La fiecare nivel:

- un thread **producător** pune în `BlockingCollection<Pereche>` (cu capacitate limitată) descrierile perechilor: `(st, mij, dr)` = intervalele `[st, mij)` și `[mij, dr)`; la final apelează `CompleteAdding()`;
- **K = 4 consumatori** iau perechile din coadă (`GetConsumingEnumerable`) și le interclasează în vectorul destinație, în intervale disjuncte, deci nu e nevoie de `lock`;
- threadul principal face `Join` pe toți, copiază intervalul fără pereche (dacă există) și apoi schimbă rolurile vectorilor sursă/destinație.

p = 1 producător + 4 consumatori = **5 threaduri**. Referința Ts este același algoritm (blocuri + interclasare pe perechi), executat secvențial. Timpul paralel include pornirea și terminarea threadurilor, operațiile pe coadă, copierea și alocarea vectorului auxiliar. Copiile vectorului nesortat se fac înainte de cronometrare; pentru fiecare configurație: o rulare de încălzire + 5 măsurări, media aritmetică.

Codul sursă (Program.cs)

```
using System;
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Threading;

namespace ConsoleApplication1
{
    // descrierea unei perechi de interclasat: [st, mij) si [mij, dr)
    struct Pereche
    {
        public int st, mij, dr;
        public Pereche(int st, int mij, int dr) { this.st = st; this.mij = mij; this.dr = dr; }
    }

    class Program
    {
        static int B = 512;           // dimensiunea unui bloc initial (constanta)
        static int K = 4;             // numarul de threaduri consumatoare (constant)
        static int P = K + 1;         // p = 1 producator + K consumatori

        static int[] sursa, dest;     // vectorii de pe nivelul curent
        static BlockingCollection<Pereche> coada;

        // ---------- Merge Sort secvential pe un interval [st, dr) ----------
        static void MergeSort(int[] a, int[] aux, int st, int dr)
        {
            if (dr - st < 2) return;
            int mij = (st + dr) / 2;
            MergeSort(a, aux, st, mij);
            MergeSort(a, aux, mij, dr);
            Interclaseaza(a, aux, st, mij, dr);
            Array.Copy(aux, st, a, st, dr - st);
        }

        // interclaseaza a[st..mij) cu a[mij..dr) in d[st..dr)
        static void Interclaseaza(int[] a, int[] d, int st, int mij, int dr)
```

```
        {
            int i = st, j = mij, k = st;
            while (i < mij && j < dr)
            {
                if (a[i] <= a[j]) d[k++] = a[i++];
                else d[k++] = a[j++];
            }
            while (i < mij) d[k++] = a[i++];
            while (j < dr) d[k++] = a[j++];
        }

        // sorteaza fiecare bloc de B elemente cu Merge Sort secvential
        static void SorteazaBlocuri(int[] a, int[] aux)
        {
            for (int st = 0; st < a.Length; st += B)
                MergeSort(a, aux, st, Math.Min(st + B, a.Length));
        }

        // ---------- Referinta: Merge Sort secvential cu interclasare pe perechi ----------
        static int[] SortareSecventiala(int[] v)
        {
            int n = v.Length;
            int[] a = v, d = new int[n];
            SorteazaBlocuri(a, d);
            for (int w = B; w < n; w *= 2)
            {
                int st = 0;
                for (; st + w < n; st += 2 * w)
                    Interclaseaza(a, d, st, st + w, Math.Min(st + 2 * w, n));
                if (st < n) Array.Copy(a, st, d, st, n - st);   // intervalul fara pereche
                int[] t = a; a = d; d = t;                        // schimbam rolurile
            }
            return a;
        }

        // ---------- Varianta paralela: producator + K consumatori pe fiecare nivel ----------
        static int nCurent, wCurent;

        static void Producator()
        {
            for (int st = 0; st + wCurent < nCurent; st += 2 * wCurent)
                coada.Add(new Pereche(st, st + wCurent, Math.Min(st + 2 * wCurent, nCurent)));
            coada.CompleteAdding();       // semnalam finalul listei
        }

        static void Consumator()
        {
            // se blocheaza cand coada e goala si se termina cand coada e inchisa si golita
            foreach (Pereche p in coada.GetConsumingEnumerable())
                Interclaseaza(sursa, dest, p.st, p.mij, p.dr);
        }

        static int[] SortareParalela(int[] v, int capacitate)
        {
            int n = v.Length;
            sursa = v;
            dest = new int[n];
            SorteazaBlocuri(sursa, dest);
            nCurent = n;

            for (int w = B; w < n; w *= 2)
            {
                wCurent = w;
                coada = new BlockingCollection<Pereche>(capacitate);

                Thread prod = new Thread(Producator);
                Thread[] cons = new Thread[K];
                prod.Start();
                for (int t = 0; t < K; t++)
                {
                    cons[t] = new Thread(Consumator);
```

```
                    cons[t].Start();
                }
                prod.Join();
                for (int t = 0; t < K; t++)
                    cons[t].Join();       // asteptam consumatorii inainte de nivelul urmator

                // intervalul fara pereche se copiaza inainte de a schimba rolurile vectorilor
                int fara = (n / (2 * w)) * (2 * w);
                if (fara + w >= n && fara < n) Array.Copy(sursa, fara, dest, fara, n - fara);

                int[] tmp = sursa; sursa = dest; dest = tmp;
                coada.Dispose();
            }
            return sursa;
        }

        // ---------- masurare ----------
        static bool Egale(int[] x, int[] y)
        {
            if (x.Length != y.Length) return false;
            for (int i = 0; i < x.Length; i++) if (x[i] != y[i]) return false;
            return true;
        }

        static int[] Genereaza(int n, int maxVal, int seed)
        {
            Random r = new Random(seed);
            int[] v = new int[n];
            for (int i = 0; i < n; i++) v[i] = r.Next(maxVal);
            return v;
        }

        // media a 5 masurari, dupa o rulare de incalzire; copiile se fac inainte de cronometrare
        static double Masoara(int[] original, int capacitate)
        {
            int rulari = 5;
            int[][] copii = new int[rulari + 1][];
            for (int i = 0; i <= rulari; i++) copii[i] = (int[])original.Clone();

            Stopwatch sw = new Stopwatch();
            double total = 0;
            for (int i = 0; i <= rulari; i++)
            {
                sw.Restart();
                if (capacitate == 0) SortareSecventiala(copii[i]);
                else SortareParalela(copii[i], capacitate);
                sw.Stop();
                if (i > 0) total += sw.Elapsed.TotalMilliseconds;   // i = 0 este incalzirea
            }
            return total / rulari;
        }

        static void Main(string[] args)
        {
            int[] lungimi = { 1000000, 4000000 };
            int[] capacitati = { 1, 8, 64 };

            Console.WriteLine("Merge Sort cu BlockingCollection: bloc B = {0}, consumatori K =
{1}, p = {2}", B, K, P);
            Console.WriteLine("Procesoare logice: {0}", Environment.ProcessorCount);

            // verificare fata de referinta secventiala, inclusiv cu valori duplicate
            foreach (int n in lungimi)
                foreach (int maxVal in new int[] { int.MaxValue, 100 })
                {
                    int[] v = Genereaza(n, maxVal, 7);
                    int[] refr = SortareSecventiala((int[])v.Clone());
                    int[] copieSortata = (int[])v.Clone();
                    Array.Sort(copieSortata);
                    bool ok = Egale(refr, copieSortata);
                    foreach (int c in capacitati)
```

```
                        ok = ok && Egale(SortareParalela((int[])v.Clone(), c), refr);
                    Console.WriteLine("Verificare n = {0}, valori in [0, {1}): {2}", n,
                        maxVal == int.MaxValue ? "2^31-1" : maxVal.ToString(), ok ? "corect" :
"GRESIT");
                }

            Console.WriteLine();
            Console.WriteLine("n;capacitate;Ts(ms);Tp(ms);S;E;C(ms)");
            foreach (int n in lungimi)
            {
                int[] v = Genereaza(n, int.MaxValue, 11);
                double ts = Masoara(v, 0);
                foreach (int c in capacitati)
                {
                    double tp = Masoara(v, c);
                    double s = ts / tp;
                    Console.WriteLine("{0};{1};{2:F1};{3:F1};{4:F2};{5:F2};{6:F1}", n, c, ts, tp,
s, s / P, P * tp);
                }
            }
        }
    }
}
```

Execuția programului

Rulat pe MacBook Air (Apple M2, 8 procesoare logice), .NET 8, configurația Release. Ieșirea programului:

```
Merge Sort cu BlockingCollection: bloc B = 512, consumatori K = 4, p = 5
Procesoare logice: 8
Verificare n = 1000000, valori in [0, 2^31-1): corect
Verificare n = 1000000, valori in [0, 100): corect
Verificare n = 4000000, valori in [0, 2^31-1): corect
Verificare n = 4000000, valori in [0, 100): corect

n;capacitate;Ts(ms);Tp(ms);S;E;C(ms)
1000000;1;75,2;53,8;1,40;0,28;269,1
1000000;8;75,2;52,3;1,44;0,29;261,4
1000000;64;75,2;51,8;1,45;0,29;259,1
4000000;1;326,5;217,0;1,50;0,30;1085,1
4000000;8;326,5;215,7;1,51;0,30;1078,5
4000000;64;326,5;213,7;1,53;0,31;1068,4
```

Verificarea compară rezultatul paralel cu referința secvențială (și referința cu `Array.Sort`), pentru valori oarecare și pentru valori din [0, 100), adică cu multe duplicate. Toate cazurile sunt corecte.

Rezultate (B = 512, K = 4 consumatori, p = 5)

n	capacitate coadă	Ts (ms)	Tp (ms)	S = Ts/Tp	E = S/p	C = p·Tp (ms)
1.000.000	1	75,2	53,8	1,40	0,28	269,1
1.000.000	8	75,2	52,3	1,44	0,29	261,4
1.000.000	64	75,2	51,8	1,45	0,29	259,1
4.000.000	1	326,5	217,0	1,50	0,30	1085,1
4.000.000	8	326,5	215,7	1,51	0,30	1078,5
4.000.000	64	326,5	213,7	1,53	0,31	1068,4

Interpretare

- Accelerarea e mică (S ≈ 1,4–1,5 față de p = 5). Sortarea blocurilor de 512 elemente se face secvențial (cum cere tema), iar asta înseamnă 9 din cele ~20 de niveluri ale sortării pentru n = 1.000.000. Doar interclasările sunt paralele.
- Pe ultimele niveluri sunt puține perechi (pe ultimul nivel una singură), deci lucrează un singur consumator și ceilalți așteaptă la coadă.
- La n = 4.000.000 accelerarea e puțin mai mare, pentru că sunt mai multe niveluri de interclasare (partea paralelă e mai mare).
- Capacitatea 1 e ușor mai lentă: producătorul se blochează după fiecare Add până ia un consumator perechea. Între 8 și 64 aproape nu e diferență, pentru că producătorul pune doar descrieri mici (3 întregi), mult mai repede decât se fac interclasările.
- Eficiența e ≈ 0,3 și costul C ≈ 3,3–3,6 × Ts: producătorul stă aproape tot timpul (e numărat în p), iar consumatorii așteaptă pe nivelurile cu puține perechi.

2. IS – Schema fluxului perechilor

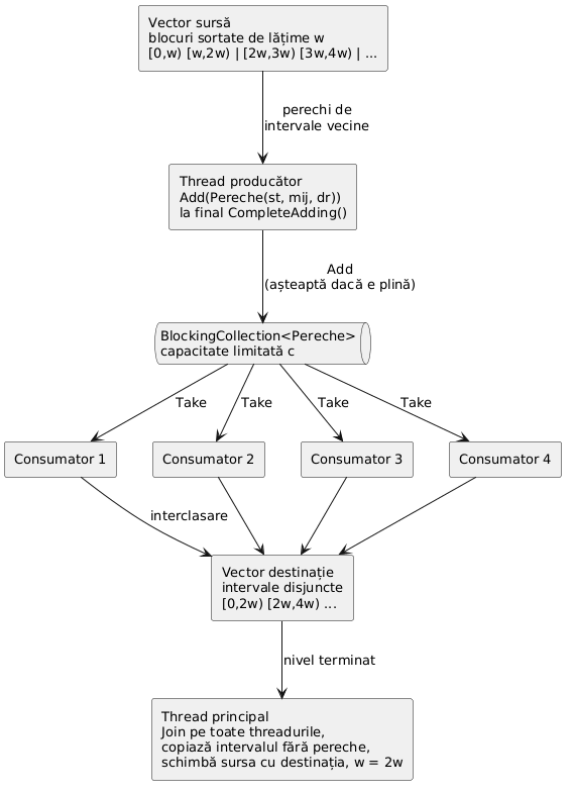


Fig. 1 – Fluxul perechilor la un nivel: producător → coadă cu capacitate limitată → consumatori → intervale disjuncte din vectorul destinație

3. IS – Pseudocodul execuției unui nivel

```
SORTARE(v, n, B, K, c):
    sorteaza fiecare bloc [i*B, min((i+1)*B, n)) cu Merge Sort secvential
    sursa <- v ; dest <- vector nou de n elemente
    w <- B
    cat timp w < n:
        NIVEL(sursa, dest, n, w, K, c)
        w <- 2*w
    rezultatul este in sursa

NIVEL(sursa, dest, n, w, K, c):
    coada <- BlockingCollection noua, cu capacitatea c
    porneste PRODUCATORUL:
        pentru st = 0; st + w < n; st = st + 2*w:
            coada.Add(Pereche(st, st + w, min(st + 2*w, n)))   // asteapta daca coada e
plina
        coada.CompleteAdding()                                  // finalul listei
    porneste K CONSUMATORI; fiecare executa:
        pentru fiecare p luat din coada (Take):                 // asteapta daca coada e
goala
            interclaseaza sursa[p.st, p.mij) cu sursa[p.mij, p.dr) in dest[p.st, p.dr)
        // iese cand coada e inchisa si golita
    Join(producator) ; Join(consumator 1..K)
    st <- (n div 2w) * 2w
    daca st < n si st + w >= n:                                 // interval fara pereche
        copiaza sursa[st, n) in dest[st, n)
    interschimba sursa si dest
```

4. IS – Diagrama de stări a consumatorului

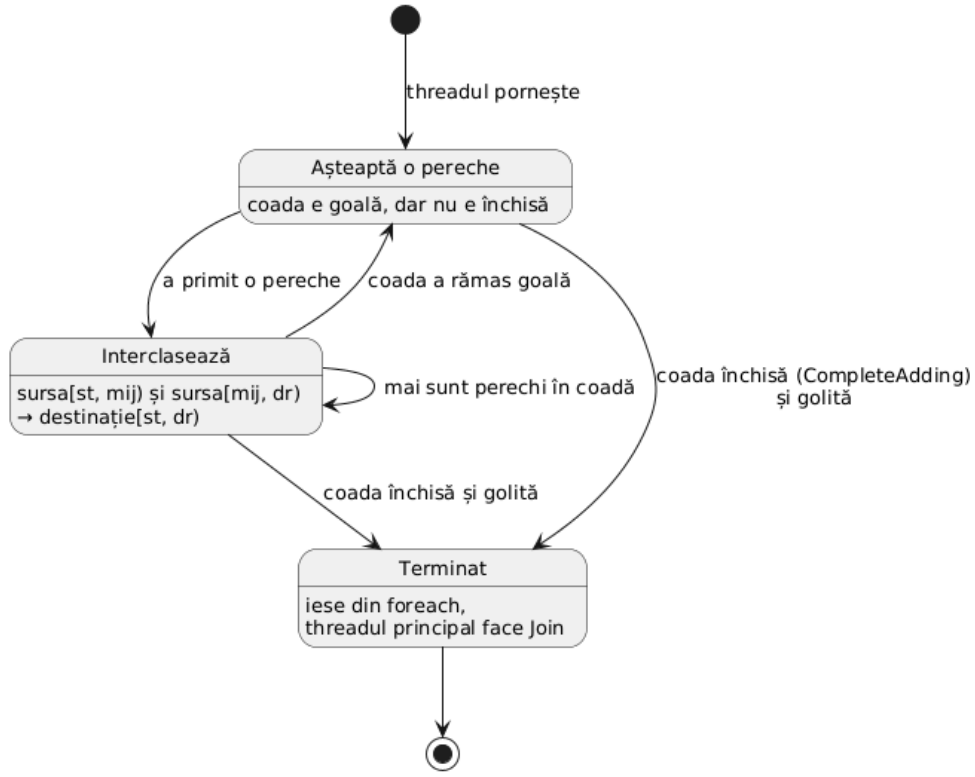


Fig. 2 – Consumatorul așteaptă când coada e goală, interclasează când primește o pereche și se termină după închiderea și golirea cozii

Diagramele au fost realizate online, cu PlantUML (plantuml.com).